Shape sensitivities of a DSMC flow

The derivation with its implementation: frame for frame with the presentation deck, plus the code of each step

main, 2026-10-09

Outline

1. The problem: the Boltzmann equation on a body-dependent domain; a windowed drag objective
2. DSMC as a Markov chain, and every place the geometry enters it
3. The value function (adjoint), carried on the particles and pulled back through moves and collisions
4. The first variation of one step: pathwise, hit-reward, score, boundary and operator terms
5. The seven terms of the shape gradient, one by one
6. Assembly, the optimiser, and what is exact and what is approximate

Notation: step k, time step Δt, F_{num} molecules per simulator particle, superscript m = the state after the move of step k (before its collisions).

Code. Mathematical reference note `docs/math/ensemble_adjoint.tex`; validation history and findings: `docs/math/shape_validation.tex`. Entry points: `examples/ensemble_mesh.py` (one seed's gradient), `examples/mesh/optimize.py`, `examples/mesh/optimize_squash.py` (optimisers). Library: `jax_dsmc/shape.py` (shape terms), `jax_dsmc/ensemble_device.py` (value backward, fits), `jax_dsmc/mesh/levelset.py` (body).

The kinetic problem

Flow domain $\Omega(\theta) = \text{box} \setminus B(\theta)$; the body is a C^1 level set

$$B = \{x : \phi(x; c) < 0\}, \qquad \phi(x; c) = \sum_j c_j B_j(x), \qquad n = \nabla\phi / |\nabla\phi|,$$

B_j cubic B-splines on knots of spacing h_ϕ, n pointing into the flow. Boltzmann equation for the one-particle density $f(t, x, v)$ in $\Omega(\theta)$:

$$\partial_t f + v \cdot \nabla_x f = Q(f, f),$$

inflow (drifting Maxwellian flux) at $x = 0$, outflow at $x = L$, specular y walls, and on the body, for $v \cdot n > 0$,

$$|v \cdot n|\, f(x, v) = \int_{v' \cdot n < 0} K(v' \to v;\, n)\, |v' \cdot n|\, f(x, v')\, \mathrm{d}v', \quad K = (1 - a)\, \delta(v - R_n v') + a\, q_w(v \mid n),$$

$R_n v' = v' - 2(v' \cdot n)\, n$; q_w the diffuse half-Maxwellian of the wall. A coefficient change moves the surface along its normal by $V_n = -\delta\phi / |\nabla\phi| = -\sum_j \delta c_j B_j / |\nabla\phi|$: every control is a normal displacement, chained through $c(\theta)$.

Code. `mesh/levelset.py` (`LevelSet`, `value`, `hit_body`; `levelset_h` = 0.0625, 16 samples). Controls: `ensemble_mesh.levelset_coef`, `PARAMETERS` (`radius`, `centre_x`, `mode2`, `width`, `squash_y`), or all coefficients (`optimize.py`). Config `acc` = a. Cases: `examples/mesh/forward.py::CASES`.

The objective: windowed drag

Mean x-force on the body over a window of W steps:

$$D(\theta) = \frac{1}{W\Delta t}\int_{t_1}^{t_2}\int_{\partial B}\int_{v\cdot n<0} m\left(v - v^{\mathrm{out}}(v)\right)_x |v\cdot n|\, f\, \mathrm{d}v\, \mathrm{d}S\, \mathrm{d}t.$$

In DSMC it is the expectation of a sum of *per-hit rewards*:

$$D = \mathbb{E}\Big[\sum_{k\in\text{window}}\ \sum_{\text{body hits } j \text{ of step } k} r_{k,j}\Big], \qquad r_{k,j} = \frac{F_{\mathrm{num}}}{W\Delta t}\, m\left(v^{\mathrm{in}}_{k,j} - v^{\mathrm{out}}_{k,j}\right)_x.$$

For a diffuse hit v^{out} is the drawn re-emission velocity.

Squash study: $J(s) = D(s) - \lambda_A\, A(s)$, an ellipse with semi-axes R and sR, $A(s) = \pi R^2 s$ exact, λ_A fixed so that the free-molecular optimum is $s^* = 0.35$. Only $\mathrm{d}D/\mathrm{d}s$ needs the adjoint. No state reward, no terminal reward.

Code. `ensemble_mesh._drag_reward` (per hit; window = the last `window` = 200 of `steps` = 300 captured steps after `warmup` = 700), `objective_value` (the same sum from the surface tallies: the reported D). `optimize_squash.reference` (free-molecular D, $\mathrm{d}D/\mathrm{d}s$, $\mathrm{d}A/\mathrm{d}s$), `seed_gradient`; history rows `dD_ds`, `J_prime`.

DSMC as a Markov chain

State $Z_k = \{(x_i, v_i, a_i)\}_{i=1}^{N}$ over fixed slots (a = alive). One step:



Common random numbers. Every stage draws from a key fixed by (seed, step, stage), so with the key fixed each stage is a deterministic function of its input: a run can be recorded and replayed exactly, and two geometries see the same random numbers.

Where θ enters the forward

(a) the move: hit times, hit points, normals, specular $v^{\mathrm{out}} = R_n v^{\mathrm{in}}$, bounce chains;

(b) the per-hit reward itself, through $v^{\mathrm{out}}(n)$;

(c) the diffuse re-emission law $q_w(\cdot \mid n)$;

(d) the hit set: which particles hit, and in which step;

(e) the collisions: NTC's attempt count carries the cut-cell flow volume $V_c(\theta)$.

Code. `simulate._step`; `simulate.run(capture=True)` → `EventRecord` (post-move `x`, `v`, `alive`, `born_face`, `PairRecord` per attempt slot, `WallEvents`); `rng.key_for`(κ, k, stage); config `record="lean"`.

NTC collisions and the cut-cell volume

In cell c with N_c particles and flow volume V_c, the attempt count is a stochastic rounding of

$$\bar{n}_{\mathrm{att},c} = \tfrac{1}{2}\, N_c(N_c-1)\, \Sigma_c\, \Delta t\, F_{\mathrm{num}}/V_c,$$

each attempt draws a pair uniformly and accepts it with probability $q(g)/\Sigma_c$, $q(g) = \sigma_T(g)\, g$ the VSS rate, Σ_c a majorant; an accepted pair scatters with the VSS law. Hence

$$\Pr\{\text{pair } (i,j) \text{ collides in step } k\} \approx \frac{q(g_{ij})\, \Delta t\, F_{\mathrm{num}}}{V_c} \quad \propto \quad \frac{1}{V_c(\theta)},$$

and a particle's expected collisions per step are $\nu \Delta t \approx (2\bar{n}_{\mathrm{att},c}/N_c)\, \mathbb{E}[q/\Sigma_c]$.

$V_c(\theta) = |c \cap \Omega(\theta)|$: a cut cell's collision probabilities depend on the shape. A cell that the body splits into disconnected flow regions gets one collision cell per region.

$\mathrm{Kn} = \lambda_{\mathrm{mfp}}/2R = 0.5$ on the squash case: $\nu \Delta t \approx 2.5\,\%$ per step; free-molecular: 0.

Code. `collide/ntc.py`: `attempt_count` (stochastic rounding), `certified_majorant`; `collide/vss.py`: `collision_rate`, `scatter`. Cut volumes by stratified sub-sampling (`GridSpec.volume`); split cells (`collision_cell`, PSEUDOCODE 4.5). Config `knudsen` (`knudsen_density`), `a_max` (attempt slots per cell and step; 64 at Kn 0.5).

The value function

Value of one particle at the post-move instant of step k:

$$\lambda_k(x,v) = \frac{\delta D}{\delta f_k^m(x,v)} = \mathbb{E}\big[\text{rewards collected from } t_k^m \text{ on } \big|\ \text{one particle at } (x,v)\big].$$

For a Boltzmann gas it solves the adjoint equation, backward in time,

$$-\partial_t\lambda - v\cdot\nabla_x\lambda = \mathcal{L}_f^*\lambda + r, \qquad (\mathcal{L}_f^*\psi)(v) = \int f(w)\int q\left[\psi(v')+\psi(w')-\psi(v)-\psi(w)\right]\mathrm{d}\sigma\,\mathrm{d}w,$$

with, at a specular wall, $\lambda(x_w, v^{\mathrm{in}}) = r(v^{\mathrm{in}}) + \lambda(x_w, R_n v^{\mathrm{in}})$, and at a diffuse wall the expectation over the re-emission.

Particle-carried form. The PDE is never solved. One value per particle, at that particle's own state, is carried backward through the recorded run:

$$\mu_{k+1} \xrightarrow{\text{collision pull-back}} \lambda_k^m \xrightarrow{\text{move pull-back}} \mu_k,$$

μ_k the values at the start of step k, λ_k^m at its post-move state.

Code. `ensemble_device.backward_virtual` → `Backward.lam` ($M \times N$: λ_k^m per captured step and slot). The objective's value and the timing-reward value (along-flight identity) run as one two-column sweep: `ensemble_mesh._backwards` (`columns_out=True`).

Backward through the move

A particle that survives the move of step k keeps its value and collects the rewards of the hits it made during the move:

$$\mu_{k,i} = \mathbf{1}\left[a^m_{k,i} \wedge \neg\, \mathrm{born}_{k,i}\right]\left(\lambda^m_{k,i} + \sum_j r_{k,j,i}\right).$$

- Specular walls are deterministic maps: the value moves with the particle.
- A diffuse wall is a Markov kernel; its realised re-emission is a sample of it, so the one carried value estimates $\mathbb{E}[\lambda(\text{after}) \mid \text{before}]$ without bias (tower property), however many bounces the step has.
- Particles absorbed or leaving the box carry 0; a particle born in step k carries nothing to t_k (it did not exist).

Code. `ensemble_device._move_pullback`, `_step_pullback_virtual`; survivors `alive & (born_face < 0)`; per-hit rewards from `shape.replay` (`HitRewards.totals()`, one total per slot and step).

Backward through the collisions: virtual partners

The collision step applies $e^{\Delta t\,\mathcal{L}^*}$ to the values; it is applied in S substeps. In each, every particle i of cell c draws K partners j_κ uniformly from the other particles of c, and fresh VSS angles:

$$\lambda_i \mathrel{+}= \sum_{\kappa=1}^{K} w_{i\kappa}\left[\Lambda(v_i') + \Lambda(v_{j_\kappa}') - \Lambda(v_i) - \Lambda(v_{j_\kappa})\right], \qquad w_{i\kappa} = \frac{q(g_{i\kappa})}{\Sigma_c}\,\frac{2\,n_{\mathrm{att},c}}{N_c}\,\frac{1}{KS},$$

Λ one smooth fit of the current values in the cell (next frames). The weight is the expected number of collisions of i in the step (attempts per particle $\times$ acceptance) spread over KS samples; all four values come from one fit, so their errors cancel.

- Substep s fits on the velocities as they stood at that point of the NTC sweep ("staged" nodes, reconstructed from the record's slot order); a particle whose velocity changed across a substep boundary moves its value through the fit.
- Splitting error $O(\Delta t/S)$; noise far below one realised pair per collision.

The collision increment of slot i in step k, $c_{k,i} = \sum_{\text{substeps}} \sum_\kappa w_{i\kappa}\Phi_{i\kappa} \approx \Delta t\,(\mathcal{L}^*\lambda)_i$, is kept: the along-flight, order and cut-volume terms use it.

Code. `ensemble_device._substep`, `_partner_pass`, `staged_velocities`, `_boundary_move`; `ensemble_backward.configure("virtual", S, K, "staged")`, default $(S, K) = (4, 8)$, collisionless runs $(1, 2)$. `backward_virtual(collision_out=True)` → `Backward.collision` ($c_{k,i}$, f32, objective column) and `Backward.cell_collision` (per-cell sums, f64). Stability guard: rms λ growth $\le$ `GROWTH_LIMIT` $= 100$ (`ensemble_inlet_shock.device_backward`).

The value field of the collision pull-back: one polynomial per cell

The pull-back needs $(3+2K)N$ field values per substep (the node, two ghosts per virtual partner, the boundary move). In the scaled coordinates $\tilde{p} = (x/\Delta x,\ v/v_{\mathrm{s}})$, $v_{\mathrm{s}} = \sqrt{kT/m}$, each collision cell c gets **one** least-squares fit of its nodes' carried values, at offsets $o = \tilde{p} - \bar{p}_c$ from the node mean:

$$\phi(o) = \big(1,\ o_x,\ o_v,\ \{o_{v,a}o_{v,b}\}_{a\le b},\ o_x^2\big), \qquad \Big(\sum_{n\in c}\phi_n\phi_n^{\top} + \rho\,\bar{d}\,\mathrm{diag}(0,1,\dots,1)\Big)\beta_c = \sum_{n\in c}\phi_n\lambda_n,$$

and every query in c reads $\Lambda(x_q, v_q) = \phi(\tilde{p}_q - \bar{p}_c)\cdot\beta_c$.

- **Order guard**: only the columns of the orders whose cumulative count is $\le n_c/1.5$ (14 columns in 2-D need 21 nodes; else a plane, else the cell mean).
- **Next to a surface** (λ jumps across $v\cdot n = 0$): three fits per cell – nodes leaving, approaching ($v\cdot n_c \gtrless 0$, n_c the cell's mean nearest-surface normal) and all; a query takes its side's fit, the unsplit one when no surface is near, its side is empty, or the cell's normals disagree ($|\sum n_i|/\#n_i < 0.9$: corners, opposed faces).
- Exact for any field quadratic in v (the per-cell fit reproduces its own monomials).

Code. `jax_dsmc/cell_fit.py`: `fit`, `fit_split`, `cell_normals`, `evaluate`; `ensemble_device._field` / `_query` (`_Hybrid` for the optional per-query fallback). `ensemble_backward.CELL_BASIS = "cell:quad:adapt1.5:align0.9:both"` (default of `configure` and every example's `--backward-basis`; `quadratic_full` = the per-query fit). Tests `tests/test_cell_fit.py`; record docs/PSEUDOCODE.md 9.7.2.

Why one fit per cell, and why the guard

- **The collision term sees only the non-invariant part of λ**: for $\lambda \in \mathrm{span}(1, v, |v|^2)$ every four-term $\Lambda(v_i') + \Lambda(v_j') - \Lambda(v_i) - \Lambda(v_j)$ vanishes. What the pull-back needs is the cell population's v-structure beyond the invariants – a property of the cell, not of a neighbourhood of each query.
- The per-query fit over the k nearest nodes depended on k ($k = 32 \to 64$: $+1.6 \pm 0.7\,\%$ of $\mathrm{d}D/\mathrm{d}s$, 12 seeds; flat from $k = 64$, where it spans the whole cell and its weights only range over $[\frac{1}{2}, 1]$) and cost $\propto k$. The per-cell fit has no k: C fits instead of $19N$.
- A cell's values are mostly noise (the cell mean predicts held-out nodes at the noise floor). An *underdetermined* quadratic in a sparse cell answers the partner ghosts with extrapolated noise, and the backward recursion compounds it: the unguarded per-cell quadratic is unstable (one seed in 12: $+97\,\%$). The guard bounds the columns by the nodes.

Measured (Kn 0.5 squash vs CRN FD): at production density (16 seeds) guarded per-cell $-3.7 \pm 0.8\,\%$, per-query $k = 32$ $-3.4 \pm 0.8\,\%$, a cubic-in-v guarded fit $-0.7 \pm 0.8\,\%$; a seed $1100 \to 46$ s. Stable on the optimiser's complicated shapes, where the per-query one-sided fit diverged.

Code. Evaluation of every interpolant tried (`mean`, `lin`, `quad`, `quadfull`, `cubicv`, `cubic`, `adapt`, `loo`, `pool`, `rb`, split rules, fallbacks): validation deck section V; `scratch/cellfit/`.

The per-query fit: the shape gradient's slopes

The shape terms need the slopes $\nabla_x\lambda$, $\nabla_v\lambda$ at reflected states, with node sets chosen per query (next frame). For a query (x_q, v_q): the k nearest admissible nodes in $\tilde{p}$, weights $\omega_n = 1/\big(1 + (d_n/d_k)^2\big)$, the same quadratic design at $\delta = \tilde{p}_n - \tilde{p}_q$:

$$\Big(\sum_n \omega_n \phi_n \phi_n^\top + \rho\,\bar{d}\,\mathrm{diag}(0, 1, \ldots, 1)\Big)\beta = \sum_n \omega_n \phi_n \lambda_n \qquad (\bar{d}\text{: mean diagonal; } \rho = 10^{-8}),$$

$$\Lambda = \beta_0, \qquad \nabla_x\Lambda = \beta_{1:1+D}/\Delta x, \qquad \nabla_v\Lambda = \beta_{1+D:4+D}/v_{\mathrm{s}}.$$

- Pooled over the 3^D block ($k = 96$): the slopes need more nodes than a value.
- The slopes are the weak link: the value field is discontinuous next to a body (next frame).

Code. `ensemble_device.node_gradients` → `cell_query` → `field_query` (value, ∇_x, ∇_v), `_design` (`basis="quadratic_full"`: 14 columns in 2-D), `_spd_solve` (unrolled Cholesky), `RIDGE_REL` $= \rho = 10^{-8}$; `cell_nodes` (`npc_max`). Until 2026-10-11 this fit ($k = 32$) was also the collision pull-back's value field.

Which nodes: functions of the current state only

Next to a body the post-move population splits: particles about to hit carry their imminent hit's reward, particles that just left do not. λ jumps across that boundary, and a polynomial fitted across it has a wrong slope. The shape-gradient fits therefore use, for a query whose last hit had normal n at x_q:

1. **one-sided**: only nodes leaving that surface, $v \cdot n > 0$;
2. **pooled**: the $k = 96$ nearest such nodes from the 3^D block of cells around the query's cell (variance);
3. **half-space**: only nodes with $(x_{\text{node}} - x_q) \cdot n \geq -\frac{1}{4}\Delta x$ (a body thinner than the block would otherwise put its far side in the pool).

Rule: stratify by the current state, never by the future. With $\lambda = P\lambda_1 + (1-P)\lambda_0$ (P the probability of some future event),

$$\nabla\lambda = P\nabla\lambda_1 + (1-P)\nabla\lambda_0 + \nabla P\,(\lambda_1 - \lambda_0);$$

a fit inside classes defined by the future event estimates the first two terms only. Classes defined by the present state ($v \cdot n$, position) keep all three.

Code. `ensemble_device.node_gradients(direction=, pool=, halfspace=)`, `pooled_tables`, `_halfspace_length`. `shape.VARIANTS`: the default `along_pool_plane` = (pool 1, $k = 96$, along-flight, half-space 0.25 cells), in `ensemble_gradient`, `optimize.py`, `optimize_squash.py`. Config `value_class="none"` (the future-class option `"hit_ahead"` is kept for diagnostics only and warns).

The first variation of one step

Write one step as $Z' = G_\theta(Z, \omega)$, randomness $\omega \sim p_\theta$, $R_\theta(Z, \omega)$ the rewards paid at the wall during the move, $V(Z')$ the value of what follows. Then

$$\frac{\mathrm{d}}{\mathrm{d}\theta}\mathbb{E}\big[R + V(Z')\big] = \underbrace{\mathbb{E}\big[\nabla V \cdot \partial_\theta G\big]}_{\text{pathwise (frozen } \omega)} + \underbrace{\mathbb{E}\big[\partial_\theta R_\theta\big]}_{\text{hit reward (frozen } \omega)} + \underbrace{\mathbb{E}\big[(R+V)\,\partial_\theta \log p_\theta(\omega)\big]}_{\text{score}}$$
$$+ \underbrace{\int_{\Sigma_\theta} [\![R + V]\!]\, u_\Sigma \,\mathrm{d}\sigma}_{\text{boundary}} + \underbrace{\mathbb{E}\big[\partial_\theta(\text{collision law) at fixed particles}\big]}_{\text{operator}},$$

Σ_θ: where G_θ jumps (hit $\to$ miss; hit into the next step), $[\![\cdot]\!]$ the jump, u_Σ its speed in θ. Summed over steps with $V = \lambda$, this gives the seven terms:

pathwise:	1 transport of the perturbed state (with the along-flight identity)
hit reward, score:	2 hit reward (tally); 3 diffuse normal
boundary:	4 hit $\leftrightarrow$ miss; 5 window edges; 6 hit/collision order
operator:	7 cut-cell volume

Code. `shape.gradient_variants` $\to$ `_gradient` (a `lax.scan` over the captured steps) $\to$ `_gradient_step`: one `jax.vjp` of the replayed wall map per step, every cotangent pulled back in one batched transpose (`_pullbacks`); output `ShapeTerms`. The value λ is a cotangent, never differentiated.

The replayed wall map

Under the fixed keys the move of step k is a pure function of the geometry:

$$(x_k^{\mathrm{post}},\ v_k^{\mathrm{post}},\ T_k,\ \mathcal{E}_k) = \mathcal{W}_\theta(x_k, v_k;\ \kappa_k),$$

T_k surface tallies, $\mathcal{E}_k$ the traced events (x^h, v^{in}, v^{out}, n, t^h per hit). It is re-run from the record and differentiated by reverse-mode AD:

- hit time: $\phi(x + tv; c) = 0$, so $\partial t^h / \partial c_j = -B_j(x^h)/(v \cdot \nabla\phi(x^h))$ (bisection for the root, one Newton step that carries the derivative);
- then $x^h = x + t^h v$, $n(x^h; c)$, $v^{\mathrm{out}} = R_n v^{\mathrm{in}}$, $x^{\mathrm{post}} = x^h + v^{\mathrm{out}}(\Delta t - t^h)$, repeated for every bounce of the step;
- grazing hits: $1/(v \cdot \nabla\phi)$ is floored at $\epsilon\,|v|\,|\nabla\phi|$ in the derivative only (the primal hit is exact): bounded per-hit weights, bias $O(\epsilon)$.

The replay must reproduce the record: any flipped discrete outcome is an error.

Code. `shape.replay` (per-hit and timing rewards, replay check; `REPLAY_TOL` $= 10^{-9}$ in `ensemble_mesh`), `shape.wall_map`, `wall_adjoint.advance(trace=True)`, `levelset.hit_body`, `levelset.regularised_quotient` (config `grazing` $= \epsilon = 0.05$).

Term 1: transport of the perturbed state (pathwise)

$$\text{pathwise}_k = \sum_{i \in \text{surv}} \nabla_x \lambda_k^m(x_i^{\text{post}}, v_i^{\text{post}}) \cdot \partial_\theta x_i^{\text{post}} + \nabla_v \lambda_k^m(x_i^{\text{post}}, v_i^{\text{post}}) \cdot \partial_\theta v_i^{\text{post}}.$$

- $\partial_\theta x^{\text{post}}$, $\partial_\theta v^{\text{post}}$: the VJP of the wall map; nonzero only for particles that hit the body in step k, so the slopes are fitted at those states only.
- $\nabla_v \lambda$: reflected velocities. On a specular body $\partial_\theta v^{\text{out}}$ is $O(1)$ per hit; the slope says how the particle's future depends on where it goes.
- $\nabla_x \lambda$: shifted hit points and hit times. Its component along the particle's own velocity comes from an identity, not from the fit (next frame).
- Only the survivors of the step contribute ($a_k \wedge a_k^m \wedge \neg\, \text{born}_k$).

Code. Cotangents from `node_gradients` on `(x_post, v_post)` in `_gradient_step`; `hit_fits=True` (`_body_hit_slots`; the same result to the last bit at $\sim$1/200 of the fitting); rows `ShapeTerms.pathwise_x`, `pathwise_v`.

$\nabla_x \lambda$ along the flight: the along-flight identity

Free flight carries $(x - sv, v)$ at time t to (x, v) at $t + s$. So

$$\lambda_t(x - sv, v) = \lambda_{t+s}(x, v) + s\,(\mathcal{L}^*\lambda)(x, v) + O(s^2), \qquad \lambda_{t+s}(x, v) = \lambda_t(x, v) + s\,T(x, v) + O(s^2),$$

$T =$ the change of the objective per unit delay of the particle's whole future (the hits that cross the window edges), and $s\,\mathcal{L}^*\lambda =$ the collisions during the extra time. Hence

$$v \cdot \nabla_x \lambda = -T - \mathcal{L}^*\lambda.$$

- T is the value of the *timing rewards* (Term 5's band rewards), carried by a second value column; $\mathcal{L}^*\lambda \approx c_{k,i}/\Delta t$ from the collision pull-back.
- The fitted $\nabla_x \lambda$ keeps its perpendicular part; its component along v is replaced:

$$\nabla_x \lambda \leftarrow \nabla_x \lambda - \frac{v \cdot \nabla_x \lambda + T + c_k/\Delta t}{|v|^2}\, v.$$

- Accuracy: NTC gives a particle one collision chance per step wherever it sits in its cell, so in the discrete scheme the $\mathcal{L}^*$ part holds to $O(\nu \Delta t)$; exact without collisions.

Code. `shape.along_flight(gx, v, t)` with `t = lam_timing + collision / dt` in `_gradient_step`; `lam_timing` = the second value column (timing rewards `Replay.timing_reward`); `collision` = `Backward.collision` (`gradient_variants(collision=)`; omitting it in a collisional run drops $\mathcal{L}^*\lambda$).

Term 2: the hit reward (tally)

The value λ_k^m starts *after* the move, so the rewards the move itself pays — the body hits of step k — enter the backward by hand, $\mu_k = \lambda_k^m + \sum_j r_{k,j}$; their derivative is this term, the value's is Term 1. The reward $r = \frac{F_{\mathrm{num}}}{W\Delta t}\, m(v^{\mathrm{in}} - v^{\mathrm{out}})_x$ depends on θ through $v^{\mathrm{out}} = R_n v^{\mathrm{in}}$ (and through v^{in} of later bounces in the same step). The surface tallies are outputs of the wall map, so this is one more cotangent:

$$\mathrm{hit}_k = \frac{\partial D}{\partial T_k} \cdot \partial_\theta T_k, \qquad \frac{\partial D}{\partial T_k} = \frac{F_{\mathrm{num}}}{W\Delta t}\, \mathbf{1}[k \in \text{window}] \text{ on the body's } x\text{-momentum.}$$

Exact given the hits; it needs no fitted field.

Example: specular cylinder of radius r, free-molecular. With impact parameter b, $F \propto \int_{-r}^{r} \cos^2\varphi \,\mathrm{d}b = 4r/3$; differentiating at fixed rays gives $\int 2b^2/r^3 \,\mathrm{d}b = 4/3$, so this term alone is F/r — the whole derivative of an isolated body, since the new grazing rays carry $\cos^2\varphi = 0$. The other terms carry the response of the flow (blockage, collisions).

Code. `ensemble_mesh._drag_tally_cotangent` × `tally_weight` in `_gradient_step`; row `ShapeTerms.pathwise_tally`.

Term 3: diffuse walls, the normal of the re-emission law

A diffuse hit re-emits from $q_w(v \mid n) = \frac{v_n}{s^2} e^{-v_n^2/2s^2} \, \frac{e^{-|v_t - u|^2/2s^2}}{2\pi s^2}$, $s^2 = kT_w/m$. Its normal moves with θ. Split the hit's future into its reward and the value after it:

$$\frac{\mathrm{d}}{\mathrm{d}n}\mathbb{E}\big[r(v^{\mathrm{out}}) + V(v^{\mathrm{out}})\big] = \underbrace{\mathbb{E}\Big[\frac{\partial r}{\partial v^{\mathrm{out}}} \cdot \frac{\partial v^{\mathrm{out}}}{\partial n}\Big]}_{\text{pathwise, reparameterised}} + \underbrace{\mathbb{E}\big[V(v^{\mathrm{out}})\, \partial_n \log q_w(v^{\mathrm{out}} \mid n)\big]}_{\text{score, value after the hit}}.$$

- Pathwise part: $v^{\mathrm{out}} = s\,(\rho\, n + z_1 t_1(n) + z_2 t_2(n)) + u$ at fixed draws; the tangent frame is parallel-transported from the primal one (bounded derivative), and the reward enters through the tally (Term 2).
- Score part: weighted by the value-to-go *after* the hit; no fitted slope. $1/v_n$ is floored at $\epsilon|v^{\mathrm{out}}|$ (bias $\approx \epsilon/2$ of the v_t/v_n part).
- Specular bodies: zero.

Code. `wall_adjoint._element_events` (tally from the traced sample), `sampling.diffuse_velocity_general` (parallel-transported frame), `shape.normal_score`, `sampling.log_diffuse_density(floor=)`; weights `to_go.post` when a tally cotangent is given; row `ShapeTerms.score`.

Term 4: hit $\leftrightarrow$ miss (silhouette band)

Along a particle's undeflected line, $\phi_{\min} = \min_s \phi(x + s\hat{v})$: < 0 it hits, > 0 it misses. A surface displacement flips the outcome where $|\phi_{\min}| < |\delta\phi|$:

$$\left.\frac{\mathrm{d}D}{\mathrm{d}\theta}\right|_{\mathrm{sil}} = -\mathbb{E}\Big[\rho(0)\,(V_{\mathrm{hit}} - V_{\mathrm{miss}})\,\frac{\partial\phi_{\min}}{\partial\theta}\Big] \approx -\frac{1}{2\varepsilon_b}\sum_{|\phi_{\min,i}|<\varepsilon_b}(V_{\mathrm{hit},i} - V_{\mathrm{miss},i})\,B_j(x_i^*)\,\frac{\partial c_j}{\partial\theta},$$

$\partial\phi_{\min}/\partial c_j = B_j(x^*)$ at the closest point (envelope theorem).

- Realised outcome: its reward-to-go. Virtual outcome: the fitted value at the virtual post-move state plus the virtual hit's reward, valued from the leaving population.
- Each passing counted once per trajectory (a band wider than one move would otherwise count it several times).
- Specular: $V_{\mathrm{hit}} \to V_{\mathrm{miss}}$ at grazing incidence, the term is $O(\varepsilon_b)$ and is left out. Diffuse: $O(1)$ — about half of a diffuse body's derivative.

Code. `shape.silhouette_cotangent`, `levelset.closest_approach`, `ensemble_device.value_query`; config `silhouette_band` $= \varepsilon_b$ (0 for specular bodies, 0.03 on the diffuse squash); row `ShapeTerms.silhouette`.

Term 5: hits that cross a window edge

The VJP differentiates a fixed assignment of hits to steps. A surface displacement shifts hit times, and a hit at the end of step k can slip into step $k+1$. For the per-hit drag this changes the objective only at the window's edges:

$$\Delta J_{k,j} = +r_{k,j} \text{ (last window step)}, \qquad -r_{k,j} \text{ (step before the window)}.$$

Boundary term: (density of hit times at the step end) $\times(-\Delta J)$, the density estimated over the hits of the w whole steps ending at each edge:

$$\text{cotangent on } t^h_{k,j}: \qquad -\frac{\pm r_{k,j}/w}{\Delta t} \quad \text{for every body hit of those steps.}$$

- Zero in expectation on a steady window, but part of the windowed objective's derivative; it is included. $w = 10$ steps: $\sim$50$\times$ the hits of a fraction-of-a-step band, at an $O(w\Delta t)$ smoothing of the edge's hit-time density.
- The same per-hit band rewards are the *timing rewards* whose value T enters the along-flight identity.

Code. `ensemble_mesh._drag_jump(width=w)`, `timing_band_setting` (a value ≥ 1 = whole steps per edge; default `timing_band` $= 10$), `shape.timing_cotangent`; row `ShapeTerms.timing`.

Term 6: the order of a hit and the step's collisions

NTC collides once per step, after the move. A body hit at the end of step k that slips into step $k+1$ is, during step k's collisions, the *incoming* particle (x_w, v^{in}) instead of the *reflected* one (x_w, v^{out}). Both futures coincide after the next move, so the objective changes by

$$J_c = c(x_w, v^{\mathrm{in}}) - c(x_w, v^{\mathrm{out}}), \qquad c = \Delta t\, \mathcal{L}^* \lambda,$$

at *every* step boundary, not only at the window's edges. The hit times of a steady flow are uniform within the step, so the density at the step end is $1/\Delta t$ and

$$\text{cotangent on } t^h : \qquad +\frac{J_c}{\Delta t} \quad \text{for every body hit.}$$

- c at the hit point: fitted from the collision increments $c_{k,i}$ of the hit's cell, on the approaching side ($v \cdot n < 0$) for v^{in} and the leaving side for v^{out}.
- Without collisions $c \equiv 0$; then Term 5 is the whole hit-step dependence.

Code. `shape.order_jump` (`ORDER_FIT` $= \{k : 32,\ \mathrm{pool} : 0\}$; `value_query(direction=`$\mp n$`)` on `Backward.collision`); cotangent `timing_cotangent(head, -jump, dt, 1.0)` on the first pass's body hits; row `ShapeTerms.order` (set when `collision` is given).

Term 7: the cut-cell volume in the collision step

Every collision probability of cell c scales as $1/V_c$ (NTC frame). The collision law is the only thing that changes, so this is a kernel-parameter (operator) term with $\partial_\theta \log q$ replaced by $\partial_\theta \log V_c^{-1}$:

$$\left.\frac{\mathrm{d}D}{\mathrm{d}\theta}\right|_V = -\sum_k \sum_c \frac{\partial_\theta V_c}{V_c}\, C_{k,c}, \qquad C_{k,c} = \tfrac{1}{2}\sum_{i\in c} c_{k,i} \quad \text{(each pair seen from both ends)},$$

$C_{k,c}$ the expected change of the objective by the collisions of cell c in step k. The geometry factor is the shape derivative of the cut volume:

$$V_c = \int_c H(\phi)\,\mathrm{d}x, \qquad \frac{\partial V_c}{\partial c_j} = \int_c \delta(\phi)\, B_j\,\mathrm{d}x = \int_{\Gamma\cap c} \frac{B_j}{|\nabla\phi|}\,\mathrm{d}s,$$

evaluated with a smoothed Heaviside $H_w(\phi) = \frac{1}{2}\big(1+\tanh(\phi/w)\big)$; each sub-region of a split cell separately. Zero without collisions.

Code. `Backward.cell_collision` (per-cell sums of $c_{k,i}$), `shape.volume_weights` ($g_c = -\sum_k C_{k,c}/V_c$), `shape.volume_cotangent` ($32^D$ samples per cell, $w = h_\phi/4$, split cells by `collision_cell`), `ensemble_mesh._volume_term` (level-set bodies, virtual pull-back); row `volume`.

Assembly

Per seed, on the level-set coefficients:

$$\widehat{\nabla_c D} = \sum_k \Big(\underbrace{\text{pathwise}_x + \text{pathwise}_v}_{1} + \underbrace{\text{tally}}_{2} + \underbrace{\text{score}}_{3} + \underbrace{\text{silhouette}}_{4} + \underbrace{\text{timing}}_{5} + \underbrace{\text{order}}_{6} \Big) + \underbrace{\text{volume}}_{7},$$

then chained to the controls by the VJP of $\theta \mapsto c(\theta)$, and averaged over seeds.

term	free-molecular, specular	free-molecular, diffuse	collisional
1 pathwise x, v	yes	yes	yes ($+\mathcal{L}^*\lambda$ along v)
2 tally	yes	yes	yes
3 diffuse normal score	0	yes	if diffuse
4 silhouette	$O(\varepsilon_b)$, off	yes	if diffuse
5 window edges	yes	yes	yes
6 hit/collision order	0	0	yes
7 cut-cell volume	0	0	yes

One gradient evaluation: forward with capture → replay → two-column value backward (with the collision increments) → one VJP scan over the captured steps → volume term → control chain.

Code. `ensemble_mesh.shape_terms` → `ensemble_gradient(variant= "along_pool_plane")`: `gradient = pathwise + score + silhouette + volume + timing + order`; `control_gradient` (`jax.vjp` of `levelset_coef`). A `newborn_hit_share` above `NEWBORN_HIT_MAX` $= 10^{-3}$ warns (`_newborn_share`).

From the gradient to the optimiser

Squash (one scalar control). Adam on $J'(s) = \widehat{\mathrm{d}D/\mathrm{d}s} - \lambda_A \, \mathrm{d}A/\mathrm{d}s$, the step's largest change of s set by the learning rate; S common-random-number seeds per iterate.

Free shape (all coefficients), at fixed area.

1. seed mean g; restrict to the coefficients within a band of the surface and smooth with a Gaussian (an H^1-type gradient that removes knot-scale noise);
2. gradient descent or Adam (Adam's normalised step is smoothed again);
3. fixed largest normal displacement per iterate ($s_{\max} = 0.05$, a tenth of a cell);
4. reinitialise ϕ to a signed distance;
5. restore the area by a uniform shift γ of ϕ: to first order the displacement becomes $V_n = \eta \, (d - \langle d \rangle_\Gamma)$, the projection of the step onto $\{\oint V_n = 0\}$, and the constraint then holds exactly (a retraction).

Code. `optimize_squash.run`, `seed_gradient`, `adam_direction` (`learning_rate` $= 0.05$); `optimize.run`, `smoothed_direction` ($\sigma = 2$ knots, band $1 + h_\phi$), `levelset.reinitialise`, `project_area` (bisection on `levelset.body_volume`). Both default to `along_pool_plane` and `timing_band` $= 10$.

What is exact, and what is approximate

piece	approximation	size / control
wall-map VJP, tally, scores	exact given the record	—
collision pull-back	S substeps, K virtual partners	$O(\Delta t/S)$; $(S,K)=(4,8)$
value field of the pull-back	one guarded quadratic per cell (split at surfaces)	$-3.7 \pm 0.8\,\%$ (Kn 0.5)
fitted slopes $\nabla\lambda$	local quadratic, state-based node sets	fit bias; k, pool, half-space
along-flight $\mathcal{L}^*$ part	continuous identity in a discrete scheme	$O(\nu\Delta t)$
grazing floor	$1/(v\cdot n)$ floored in the derivative	$O(\epsilon)$; $\epsilon = 0.05$
silhouette band	finite band density	$O(\varepsilon_b)$
window-edge band	w whole steps	$O(w\Delta t)$ smoothing
order term	uniform hit times in the step	steady flow
captured horizon	sensitivities older than the captured steps	box memory (walls)

Left out on purpose: hits by particles born in the step or leaving the box in it (monitored, 0 while the body is away from the inflow and outflow faces); the change of cell membership of a hit particle ($O(\nu\Delta t)$ below the order term); the silhouette term of specular bodies.

Validation of each piece against finite differences: the validation deck.

Code. Settings: `ensemble_backward.VIRTUAL`; `shape.VARIANTS`; `grazing`; `silhouette_band`; `timing_band`; captured `steps` (300 in production, ≥ 600 for 1 %-level free-molecular gates); `NEWBORN_HIT_MAX`.

Settings and their defaults

setting	where	default	role
`variant`	`ensemble_gradient`, optimisers	`along_pool_plane`	$\nabla_x \lambda$ estimator (`shape.VARIANTS`)
`--backward-basis`	`configure(basis=)`, CLI	`CELL_BASIS`	pull-back's value field: `cell:quad:adapt1.5:align0.9:both`
`neighbors`	config / CLI	32	k of the unpooled per-query fits (backward only with `quadratic_full`)
pool, k, half-space	`shape.VARIANTS`	1, 96, 0.25 cells	pooled shape fits
`value_class`	config	`"none"`	`"hit_ahead"`: diagnostic only, warns
`VIRTUAL`	`ensemble_backward.configure`	$(4, 8)$ staged	(S, K); $(1, 2)$ collisionless
`RIDGE_REL`	`ensemble_device`	10^{-8}	relative ridge of every fit
`grazing`	config	0.05 (level set runs)	ϵ of the $1/(v \cdot n)$ floors
`silhouette_band`	config	0 (specular), 0.03	ε_b
`timing_band`	config / CLI	10	w whole steps per window edge (< 1: step fraction)
`ORDER_FIT`	`shape`	$k = 32$, pool 0	fit of c at the hit point
volume samples, width	`volume_cotangent`	32^D, $h_\phi/4$	smoothed cut volume
`levelset_h`, samples	config	0.0625, 16	knot spacing, hit-search samples
`warmup`, `steps`, `window`	config	700, 300, 200	captured horizon and window
`REPLAY_TOL`	`ensemble_mesh`	10^{-9}	replay continuous error
`NEWBORN_HIT_MAX`	`ensemble_mesh`	10^{-3}	warning on unscored hits
`GROWTH_LIMIT`	`ensemble_inlet_shock`	100	refuse an unstable backward
`learning_rate`	`optimize_squash`	0.05	largest Δs per iterate
step, smoothing, band	`optimize`	0.05, $\sigma = 2$, $1 + h_\phi$	free-shape optimiser

Data flow of one gradient evaluation (`ensemble_mesh.shape_terms`)

1. `forward(cfg, ..., capture=True)`: warm-up, then `steps` captured steps → `EventRecord` (`M` steps × `N` slots, lean record).
2. `shape.replay`: re-runs $\mathcal{W}_\theta$ per captured step; per-hit rewards (`_drag_reward`), timing rewards (`_drag_jump`), replay error.
3. `_backwards`: one `backward_virtual` sweep with two value columns (hit rewards, timing rewards), `collision_out=True`, `columns_out=True` → `lam`, `lam_timing` (f64[M, N]), `collision` (f32[M, N]), `cell_collision` (f64[M, C]).
4. `shape.gradient_variants(lam_timing, collision=...)`: one scan over the captured steps, one VJP per step; all `VARIANTS` share the replay and the non-fitted cotangents → `ShapeTerms` per variant (incl. `order`).
5. `_volume_term`: `volume_weights` + `volume_cotangent`.
6. `control_gradient`: the VJP of `levelset_coef` for a scalar control; the optimiser of free shapes keeps the coefficient gradient.

Memory on the 4 GB workstation GPU: `fnum_scale` ×2 with 300 captured steps is the largest collisional squash case; production runs use the cluster.

Tests that pin each piece (tests/)

piece	tests
virtual pull-back	test_ensemble_device.py (host = device; quadratic invariant to 10^{-12} at zero ridge)
per-cell field	test_cell_fit.py: polynomial reproduction, least squares and slopes, order guard, empty split sides, sigma points, complex-cell fallback = per-query one-sided backward
value-form rewards	test_value_form_rewards_accumulate_backward_without_collisions
field slopes	test_field_query_returns_exact_position_slopes_of_a_quadratic_field, test_pooled_one_sided_node_gradients_are_exact_on_a_quadratic_field, test_halfspace_drops_the_nodes_behind_the_query_plane
wall-map VJP, replay	test_wall_map_vjp_matches_finite_differences_on_specular_hits, test_replay_reproduces_the_captured_forward_bitwise_and_the_vjp_runs, test_replay_mismatch_separates_discrete_divergence_from_roundoff
along-flight	test_along_flight_replaces_only_the_component_along_the_velocity
diffuse normal	test_normal_score_matches_finite_differences_of_the_density, test_diffuse_reparameterisation_in_the_normal_is_bounded_and_unbiased
silhouette	test_silhouette_term_counts_the_hit_set_change
window edges	test_timing_cotangent_selects_the_band_with_the_lost_jump, test_traced_times_are_the_hit_times
cut-volume term	test_volume_weights_sum_the_cell_collision_terms_over_the_volume, test_volume_cotangent_of_unit_weights_is_the_flow_area_derivative
level set	test_mesh_levelset.py (hit position error $O(h_\phi^4)$, $\partial t/\partial c$ against the analytic body)

Gates against finite differences are not unit tests: they are in the validation deck, with the scripts that